

EyeProtect 项目技术总述

算法依据、工具调用与实现边界

EyeProtect Project

2026 年 7 月

目录

1	项目定位	2
2	总体技术架构	2
3	功能与依据	2
3.1	屏幕使用时间统计	2
3.2	前置摄像头调用	3
3.3	人脸与眼部识别	3
3.4	眼部点位提取	4
3.5	眼部边界框计算	5
3.6	眼部中心点	5
3.7	坐标平滑	5
3.8	左右眼 ROI 裁剪	6
4	使用的工具、SDK 与开源边界	6
5	ML Kit 与 MediaPipe 的关系说明	6
6	当前输出数据	7
7	资源与生命周期策略	8
8	当前限制	8
9	后续扩展建议	9
10	参考资料	9

1 项目定位

EyeProtect 是一个部署在 Android 平板端的护眼原型项目。当前版本已经完成：

- 通过 Android 系统接口读取屏幕/App 使用时间；
- 通过 CameraX 调用前置摄像头；
- 通过 Google ML Kit Face Detection 检测人脸和左右眼位置；
- 绘制左右眼关键点、眼部边界框，并输出眼部坐标；
- 计算 FPS、检测耗时、人脸框、眼部中心点；
- 裁剪并显示左右眼 ROI；
- 在后台、锁屏、横竖屏场景下管理摄像头资源。

当前项目的眼部识别目标是“眼部位置检测”，不是虹膜检测、眼球方向判断或屏幕注视点预测。

2 总体技术架构

项目由 Android 原生能力、CameraX 视频采集、ML Kit 人脸检测和自定义规则算法组成。

Android UsageStatsManager

-> 屏幕/App 使用时间

CameraX Preview

-> 前置摄像头实时预览

CameraX ImageAnalysis

-> 实时读取视频帧

-> InputImage

-> Google ML Kit Face Detection

-> Face / FaceContour / FaceLandmark

-> 左右眼点位

-> 边界框、中心点、平滑、ROI 裁剪

-> Overlay 绘制与 Logcat 输出

3 功能与依据

3.1 屏幕使用时间统计

屏幕使用时间统计基于 Android 平台 API: UsageStatsManager。

实现依据：

- Android 系统会记录应用进入前台的使用情况；

- 应用获得“使用情况访问权限”后，可以查询一段时间内的应用使用统计；
- 项目使用 `queryUsageStats` 查询今日统计，并读取 `totalTimeInForeground` 作为使用时长。

当前代码位置：

- `app/src/main/java/com/example/eyeprotect/usage/UsageAccessHelper.kt`
- `app/src/main/java/com/example/eyeprotect/usage/ScreenUsageRepository.kt`

算法流程：

检查 Usage Access 权限

- > 未授权：跳转系统设置页
- > 已授权：`UsageStatsManager.queryUsageStats(...)`
- > 按 `packageName` 聚合 `totalTimeInForeground`
- > 按使用时长倒序展示

这部分不是视觉算法，也不是后台录屏；它是系统级使用统计读取。

3.2 前置摄像头调用

前置摄像头调用基于 AndroidX CameraX。CameraX 是 Android Jetpack 中的相机库，用于简化不同 Android 设备上的相机兼容性处理。

项目中使用两个 CameraX use case：

- **Preview**：负责把摄像头画面显示到界面；
- **ImageAnalysis**：负责实时读取帧，送入眼部检测链路。

关键实现：

```
CameraSelector.DEFAULT_FRONT_CAMERA
ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST
ImageAnalysis.Builder().setTargetResolution(Size(640, 480))
```

当前代码位置：

- `app/src/main/java/com/example/eyeprotect/camera/FrontCameraController.kt`

选择 `STRATEGY_KEEP_ONLY_LATEST` 的原因是实时检测更重视低延迟，不需要处理积压旧帧。

3.3 人脸与眼部识别

眼部识别当前基于 Google ML Kit Face Detection SDK。

项目中使用的依赖：

```
implementation("com.google.mlkit:face-detection:16.1.7")
```

需要特别说明：当前 Android 版本不是使用 Python/OpenCV/MediaPipe 实现的。Python 版本曾作为 PC 端验证方案存在，但平板端实际运行的是 CameraX + ML Kit。

ML Kit Face Detection 在项目中的作用：

- 检测单帧图像中的人脸；
- 输出人脸框；
- 输出左右眼 landmark；
- 输出左右眼 contour，即眼部轮廓点。

关键配置：

```
FaceDetectorOptions.Builder()
    .setPerformanceMode(FaceDetectorOptions.PERFORMANCE_MODE_FAST)
    .setLandmarkMode(FaceDetectorOptions.LANDMARK_MODE_ALL)
    .setContourMode(FaceDetectorOptions.CONTOUR_MODE_ALL)
    .setClassificationMode(FaceDetectorOptions.CLASSIFICATION_MODE_ALL)
    .enableTracking()
    .build()
```

当前代码位置：

- app/src/main/java/com/example/eyeprotect/vision/EyePositionAnalyzer.kt

3.4 眼部点位提取

ML Kit 返回 Face 对象后，项目优先读取：

```
FaceContour.LEFT_EYE
FaceContour.RIGHT_EYE
```

如果 contour 不可用，则使用：

```
FaceLandmark.LEFT_EYE
FaceLandmark.RIGHT_EYE
```

并基于中心点生成一个近似眼框。

算法逻辑：

```
Face
-> getContour(LEFT_EYE).points
-> getContour(RIGHT_EYE).points
-> 如果轮廓点为空，回退到 getLandmark(...)
```

3.5 眼部边界框计算

眼部边界框不是 ML Kit 直接给出的眼框，而是项目根据眼部点位计算出来的。

设眼部点集为：

$$P = [(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)]$$

边界框计算：

```
box.x1 = min(x) - padding
box.y1 = min(y) - padding
box.x2 = max(x) + padding
box.y2 = max(y) + padding
```

之后裁剪到图像范围：

```
x1 >= 0
y1 >= 0
x2 <= imageWidth - 1
y2 <= imageHeight - 1
```

当前代码位置：

- `EyePositionAnalyzer.calculateBoundingBox(...)`
- `app/src/main/java/com/example/eyeprotect/vision/BoundingBox.kt`

3.6 眼部中心点

眼部中心点基于边界框计算：

```
centerX = (box.x1 + box.x2) / 2
centerY = (box.y1 + box.y2) / 2
```

注意：当前中心点表示“眼部框中心”，不是虹膜中心，也不是视线落点。

3.7 坐标平滑

实时检测中的眼部点会轻微抖动。项目使用指数平滑降低视觉抖动。

公式：

$$\text{smoothed} = \text{previous} + \alpha * (\text{current} - \text{previous})$$

当前 `alpha` 设置为 0.35。该值越大，响应越快但抖动越明显；该值越小，画面更稳但跟随延迟更高。

当前代码位置：

- `EyePositionAnalyzer.smoothEye(...)`

3.8 左右眼 ROI 裁剪

ROI 是 Region of Interest，即感兴趣区域。项目根据平滑后的眼部边界框，从当前帧图像中裁剪左右眼小图。

流程：

```
CameraX ImageProxy
-> YUV_420_888
-> NV21
-> JPEG
-> Bitmap
-> 按旋转角度修正
-> 根据 eyeBox 扩大 1.8 倍
-> Bitmap.createBitmap(...)
```

当前代码位置：

- `app/src/main/java/com/example/eyeprotect/vision/ImageProxyBitmapConverter.kt`
- `EyePositionAnalyzer.cropEyeRoi(...)`

ROI 当前只用于显示和后续算法准备；尚未用于虹膜识别或眼球方向判断。

4 使用的工具、SDK 与开源边界

组件	用途	项目中的调用方式	开源/开放边界
Android Platform API	系统使用统计、权限、生命周期	UsageStatsManager、AppOpsManager、BroadcastReceiver	Android 平台 API, AOSP 中有大量平台代码；设备实现可能由厂商定制。
AndroidX CameraX	前置摄像头预览与帧分析	Preview、ImageAnalysis、CameraSelector.DEFAULT_FRONT_CAMERA	AndroidX/Jetpack 开源库
Google ML Kit Face Detection	人脸、眼部 landmark/contour 检测	FaceDetection.getClient(...), FaceContour.LEFT_EYE、FaceContour.RIGHT_EYE	公开 SDK；底层模型不是本项目可修改的开源模型。
Kotlin + Gradle	Android 应用开发与构建	Kotlin 源码、Gradle Android Plugin、依赖管理	Kotlin 与 Gradle 为开源工具链。
自定义规则算法	边界框、中心点、平滑、ROI	calculateBoundingBox、smoothEye、cropEyeRoi	项目自研规则代码，可完全修改。

5 ML Kit 与 MediaPipe 的关系说明

项目早期曾实现过一个 PC 端 Python 版本：

Python + OpenCV + MediaPipe Face Mesh

该版本用于验证眼部点位检测思路。但当前可安装到华为平板的 Android APK 没有运行 Python、OpenCV 或 MediaPipe Python 代码。

当前 Android APK 的实际链路是：

CameraX

```
-> ML Kit Face Detection  
-> FaceContour.LEFT_EYE / RIGHT_EYE  
-> 自定义边界框与 ROI 算法
```

因此，正式描述应写为：

本项目的 Android 端眼部位位置识别基于 Google ML Kit Face Detection SDK。系统通过 CameraX 获取前置摄像头实时视频帧，将帧输入 ML Kit Face Detector，利用其输出的人脸框、左右眼轮廓点和眼部关键点，计算左右眼边界框、中心点和 ROI 区域。

6 当前输出数据

项目当前输出的核心数据包括：

- timestamp: 时间戳;
- face_id: 人脸跟踪 ID;
- face_count: 检测到的人脸数量;
- fps: 平滑后的帧率;
- latency_ms: 单帧检测耗时;
- source_width/source_height: 分析帧尺寸;
- face_box: 人脸框;
- left_eye_box/right_eye_box: 左右眼框;
- left_eye_center/right_eye_center: 左右眼中心点。

示例：

```
{  
  "timestamp": 1783651200125,  
  "face_id": 0,  
  "face_count": 1,  
  "fps": 24.8,  
  "latency_ms": 37,
```

```
"source_width": 640,
"source_height": 480,
"face_box": {"x1": 180, "y1": 70, "x2": 460, "y2": 390},
"left_eye_box": {"x1": 245, "y1": 160, "x2": 305, "y2": 195},
"left_eye_center": {"x": 275, "y": 178},
"right_eye_box": {"x1": 340, "y1": 160, "x2": 400, "y2": 195},
"right_eye_center": {"x": 370, "y": 178}
}
```

7 资源与生命周期策略

摄像头是高占用资源，项目在工程层面实现了以下策略：

- 应用进入后台时释放摄像头；
- 应用回前台时，如果此前摄像头正在运行，则自动恢复；
- 屏幕关闭或锁屏时停止摄像头，不自动恢复；
- 用户可以点击 **Stop Camera** 手动释放摄像头；
- 横竖屏切换时保存恢复意图，避免无谓丢失状态。

当前代码位置：

- `MainActivity.onPause`
- `MainActivity.onResume`
- `MainActivity.ACTION_SCREEN_OFF` `BroadcastReceiver`
- `FrontCameraController.stopPreview`

8 当前限制

当前版本仍有明确边界：

- 不能检测虹膜中心；
- 不能判断眼球向左/向右/向上/向下；
- 不能预测屏幕注视点；
- 眼部中心点是眼框中心，不是瞳孔或虹膜中心；
- ML Kit Face Detection 的底层模型不可在本项目内训练或修改；
- 在弱光、遮挡、强侧脸、多人脸场景下检测稳定性会下降。

9 后续扩展建议

建议下一步顺序：

1. 睁闭眼检测：使用 ML Kit 的 `leftEyeOpenProbability/rightEyeOpenProbability`；
2. 眨眼事件检测：实现 OPEN -> CLOSED -> OPEN 状态机；
3. 疲劳提醒：结合闭眼时长、眨眼频率、屏幕使用时间；
4. 距离过近提醒：基于人脸框高度比例或双眼中心距离；
5. 虹膜与视线方向：评估 MediaPipe Face Landmarker / Iris 能力或独立模型；
6. 屏幕注视点预测：需要用户校准数据，不建议在无校准条件下直接实现。

10 参考资料

- Android UsageStatsManager 官方文档：<https://developer.android.com/reference/android/app/usage/UsageStatsManager>
- Android CameraX 图像分析官方文档：<https://developer.android.com/media/camera/camerax/analyze>
- AndroidX Camera 发布说明：<https://developer.android.com/jetpack/androidx/releases/camera>
- Google ML Kit Face Detection Android 文档：<https://developers.google.com/ml-kit/vision/face-detection/android>
- Google ML Kit Face Detection Concepts：<https://developers.google.com/ml-kit/vision/face-detection>
- Kotlin 官方网站：<https://kotlinlang.org/>
- Gradle 官方网站：<https://gradle.org/>